

第 1 课 · 行业地图与问题定位

AI Ops Agent 是干什么的，我们这节课的项目要解决什么问题

第 1 课 · 90 分钟（讲授 50 + 实操 30 + 小面试 10）

一、这节课的目标

三件事：第一，搞清楚 AI 应用这块到底有哪些岗位、各自干什么；第二，明确我们这个项目落在哪个位置上；第三，把项目跑起来，先看到终点长什么样。

这节课不写代码。跑通环境、看到成品、知道后面五节课要干什么，就够了。

二、口语化讲稿

引入：一个真实的问题

我先不讲概念，先讲一个场景。你是一个平台的研发，凌晨两点，群里有人 @ 你：「我的训练任务挂了，帮我看看」。你打开平台，找到那个 job，点开日志——一万两千行。你滚动鼠标，在最后几行看到一个报错。

然后你要做的事情是：判断这个报错是根因还是症状，去翻这个任务用了哪些卡、跑在哪个集群、退出码是多少、当时显存和网络是什么曲线，最后得出一个结论：「这是显存爆了，把 batch size 调小」。

整个过程可能二十分钟。而这一整套动作——查记录、读日志、看指标、对知识、下结论——恰好是大模型 agent 最擅长的那类活儿：有明确的工具可调，有明确的判断标准，有现成的领域知识。这就是我们这节课要做的东西。

这个项目的定位

一句话：给一个 job id，agent

自己去查证，然后给出结构化的诊断——根因是什么、证据是什么、该怎么修。

它做什么	它不做什么
读 job 记录、日志、指标	重启任务、改队列、改配额

它做什么	它不做什么
检索平台知识库 (runbook)	删除 checkpoint、写生产系统
给出根因和修复建议	替工程师做决定
拒绝在证据不足时作答	猜测一个「大概是这样」的答案

只读是硬约束，不是偷懒。一个能重启任务的助手，迟早会重启错的那个。

为什么讲这个项目

市面上讲 agent 的材料，绝大多数停在「调一个 API、写一个 loop、接两个工具」。那部分东西一个下午就能学会，也确实只有两百行。

但真实系统里难的不是那个循环。难的是：模型服务很贵而且很脆，同时打过来的请求超过它承载能力就会全线崩溃；一家公司可能把整个集群的资源用光；每一轮对话都在烧钱；网络会抖、模型会挂、下游会超时。这些东西加起来，才是一个能上线的 agent 系统和 demo 的区别。

所以这个项目里，agent loop 只有两百行。剩下的九千多行，全都在解决「怎么让这两百行在真实压力下活下来并且可度量」。这就是我们这六节课的主线。

四个岗位，你投哪个

「大模型应用工程师」不是一个岗位，是一族。看 JD 的时候先分清对方要哪一类。

岗位	核心技能	面试会问什么
LLM 应用工程师	提示词、工具调用、RAG、评测	「你的知识库数据从哪来、怎么保证质量」
Agent 工程师	任务规划、多步推理、状态机、工具协议	「白板写一个 agent loop」
大模型平台工程师	推理服务、并发、限流、成本、可观测	「上游被打挂了怎么办」
评测工程师	指标体系、golden set、LLM-as-judge	「怎么证明你改完变好了」

四类的交集是：都要求你会写工具调用、都要求你看得懂评测数字。所以这个项目里四类人的活儿都有。

三、动手：把项目跑起来

环境

需要 Python 3.10+。不需要 GPU、网络、API key。

```
git clone https://github.com/lianghao6/ailab-ops-copilot.git
cd ailaab-ops-copilot
pip install -e ".[dev]"
```

四步看到全套效果

```
make data      # 00000000 2 0
make test      # 159 000000 3.5 0
make demo      # 0000000000
make serve     # 0 HTTP 00000000 127.0.0.1:8080
```

跑 make demo，注意看它打印的东西。它先列出 agent 调了哪些工具，然后给出根因、证据、排除了哪些其他可能、以及修复建议。我把最后那个 ground truth 对比也打出来了——那是答案，agent 看不到，只在演示时给你看，让你确认它到底判对没有。

在 Web UI 上按一下最关键的按钮

起来服务以后，右下角有一个「kill the model service」按钮。按下去，系统会认为模型已经挂了。然后再问一次问题——注意，它**仍然给出了答案**，只是标记成 degraded。

这是这个项目我觉得最值得看的一个设计：模型挂了不等于系统挂了。工具是对本地数据的读取，打分是确定性代码，所以不调模型一样能出诊断。第 4 课我们会详细讲这个降级阶梯。

四、六节课的地图

课主题次	你会拿到什么
1 行业地图与问题定位	跑通环境，知道终点长什么样
2 Agent 原理与上下文工程	手写一个 agent loop

课主题次	你会拿到什么
3 检索与知识底座	把 runbook 检索跑通并量化
4 高并发与稳定性	压测报告 + 故障注入
5 评测与可观测	一份能回答「改完变好没有」的评测报表
6 答辩与模拟面试	完整演示 + 简历写法

第 3、4 课是压轴。它们最能体现「工程」二字，也是面试得分点最集中的地方。

五、小面试（10 分钟）

现在就用第 1 课的内容做一轮真实面试。先自己答，再看参考答案。

Q1：你为什么想做大模型应用方向？

不要答「因为火」。要答「因为我现有的某段经历，和大模型应用需要的某项能力是同一件事」。

- 错误答法：「我对 AI 很感兴趣，一直在自学。」——没有任何信息量。
- 及格答法：「我之前做过分布式训练平台的运维，处理过大量任务失败诊断。」
- 满分答法：接上句，再加上「我发现这类问题的形态和我现在学的 agent 高度一致：有明确工具、有判断标准、有现成的领域知识，而且失败率高、重复劳动多，正好适合自动化。」

Q2：你了解哪些 agent 框架？

- 不要背框架名。面试官想知道你是「用过」还是「懂」。
- 可以答：「我用过 X，但更想说的是它解决的问题——把工具调用、多轮状态、提示词模板封装起来。」

- 然后一定要接一句框架不管的事：「它不管并发、限流、成本、也不管怎么证明改完变好了。这些是我现在这个项目主要在做的。」

这一句是分水岭。只讲框架的人很多，指出框架边界的人很少。

Q3：给你一个任务，你怎么判断该不该用 agent？

- 有明确工具可调（有 API、有数据可查）
- 有明确判断标准（对错可验证，而不是「感觉答得不错」）
- 步骤数不固定（否则写死流程更稳更便宜）
- 失败可恢复（做错了能重来，不会造成不可逆后果）

反面例子：一个只有一步、且答案唯一的问题，用 agent 是纯浪费——一次函数调用就够了。

六、作业

1. 把你的数据集跑出来，在 data/generated/failures.csv 里找一条 category 是 runtime 的记录。
2. 对着那条 job 跑 ailab-ops inspect <job-id>，看它打印的完整证据链，特别注意 HYPOTHESIS SCORES 那一段——那是确定性打分，第 3 课会讲。
3. 打开 Web UI 的故障注入按钮，截图降级前后的对比。

下节课开始写代码。把你自己的种子跑出来的数据集带着，后面所有实操都用它。